AI-OS Paper: README for Finishing and Publishing

Drop this file into the repo (for example `paper/README.md`) and tick items off as you go. Written 13 Oct 2026. Deadlines and policies change, so re-check every date and rule against the official page before acting.

---

0. Where you are

- The v1 draft exists (architecture, related work, evaluation protocol). It contains **no results**.
- Every `[TBD]` and every `[verify]` in the draft must be gone before submission.
- The draft's technical claims came from your README, ARCHITECTURE.md and PR #3, **not from the source code**. They must be audited (Phase 0).
- The paper becomes publishable only when Phases 1 to 4 produce evidence for the claims. Without them it is a system description and will likely be rejected or sent to a lower-tier venue.

---

1. Decisions to make first (this week)

#	Decision	Recommendation
1	Affiliation	See section 2. Default: Independent Researcher, unless your employer gives written permission.
2	Authors	Decide who qualifies. `smalinidevi` reviewed and merged PR #3; if she contributed ideas, code or experiments, discuss authorship. Otherwise add her to Acknowledgements.
3	Title and framing	Avoid claiming to be an "operating system". Name the contribution as an orchestration or governance layer. Reviewers will compare "AI-OS" with the Rutgers AIOS paper (arXiv:2403.16971).
4	Paper type	Aim for a **systems + empirical** paper: architecture plus ablations. Do not frame it as a survey or position paper (arXiv now requires prior peer review for CS surveys and position papers).
5	Target	Plan: arXiv preprint first, then a workshop or rolling-review venue (section 5).
6	Repo licence	Add a `LICENSE` file (MIT or Apache-2.0) only **after** the employer check in section 2.

---

2. Affiliation and your employer (read before you post anything)

I am not a lawyer; use this as a checklist, and ask HR or a lawyer if any clause is unclear.

What to write

- **Default, if the work is genuinely independent:** `Independent Researcher` plus your city or country (optional) and a **personal** email. Add an ORCID iD (free) and a Google Scholar page so the name is traceable.
- **Employer as affiliation:** only if your employer approves in writing, or the work was done as part of your job. Putting a company name on a paper implies the company stands behind it.
- **Do not** use your employer's name informally (for example "Senior AI Engineer at X") in the byline as a way to look credible. It can create IP and reputation problems for you and for them.
- Affiliation is free text in the paper and on arXiv; nobody verifies it, so honesty is your responsibility.

Checklist before submission

☐ Read your **employment contract** for: invention and IP assignment (does it cover anything "related to the company's business", or only work done on company time?), moonlighting or outside-activity clauses, and publication or confidentiality clauses.
☐ Confirm the project was built on **personal time and personal equipment and accounts**: no company laptop, cloud credits, API keys, code, data, prompts or customer information.
☐ Confirm no scenario data is real client data. (The support and audit demos are synthetic; keep it that way and say so.)
☐ If there is any overlap with your employer's products or customers, **ask for a written No-Objection / publication approval** (email from HR or your manager's manager is usually enough). Keep it.
☐ If the employer approves, add this style of line in the paper: *"The author is employed by [Company]. This work was conducted independently, on personal time and equipment, and does not represent the views of the employer."* Only use the line with their consent.
☐ Use your personal email for the paper, arXiv, OpenReview and the repo.

If the employer refuses or the clause is ambiguous, stop and resolve it first; unpublishing later is hard.

AI assistance and disclosure

Your commit history lists an AI model as co-author, and this draft was written with AI help. Many venues now require a statement about AI use (ICLR 2027 reportedly mandates one; confirm on the official page). Prepare a short honest statement: what was AI-assisted (code, drafting, literature search), and that **you** verified the claims, ran the experiments and take responsibility. Do not hide it.

---

3. Work plan

Rough effort is for one person working evenings and weekends; adjust after the pilot.

Phase 0: Audit and pin (about 1 week)

- [ ] Create branch `paper/eval`; **pin the commit hash** you will evaluate (PR #3 merge is `d259651`, but pin whatever is final).
- [ ] Read `coordinator.py`, `planner.py`, `state.py`, `semantic.py`, `agents/base.py`, `mcp_gateway/`. Compare against paper Sections 3 to 5; fix every mismatch.
- [ ] Record exact values and put them in the paper: confidence threshold θ, retry count and backoff, critical vs non-critical rules, how a claim key is built, what "supersede" does in storage.
- [ ] Run the 129 tests and the offline demo on a clean machine. Record Python and LangGraph versions.
- [ ] Add per-run logging of: model id, temperature, tokens in and out, latency, tool calls. You need this for cost metrics.
- [ ] Expose temperature and seed settings. Ensure replay mode is clearly separate from live runs.

Phase 1: Fix the known weaknesses (about 2 weeks)

These give the paper something to measure (before vs after).

- [ ] **Claim identity.** Normalise numbers ("55,700" = "55700.00"), units and currency; add an alias map or embedding match for `(subject, metric)`. Keep the old literal matcher as the baseline.
- [ ] **Claim schema.** Add `as_of` (timestamp) and `scope` / `period` fields. This lets the coordinator tell a stale snapshot or a different time window from a true disagreement.
- [ ] **Confidence.** Add a consistency-based estimate (sample k=3 to 5 answers, measure agreement) and a hybrid with the verbalised score. Calibrate on a dev set (for example isotonic regression). Keep raw confidence as the baseline.
- [ ] **Recall.** Implement embedding or hybrid recall behind `FactStore.recall` for the scaling experiment.
- [ ] Enforce or remove the three unenforced grant fields (`rate_limit_per_minute`, `secret_ref`, `max_wall_clock_seconds`).
- [ ] Make every new mechanism a **config flag** so ablations are one switch each.

Phase 2: Evaluation harness (about 3 weeks)

Suggested layout:

```
eval/
  generators/conflicts.py     # seeded scenario generator with ground truth
  faults/injector.py          # wraps the tool gateway: 503, timeout, stale,
corrupted, schema change
  baselines/                  # single ReAct agent, supervisor-no-reconcile,
AutoGen group chat, debate
  metrics/                    # conflict_prf.py, calibration.py, faults.py,
memory.py, passk.py
  configs/                    # one YAML per experiment condition
  runs/                       # raw JSONL results + traces (never edit by
hand)
  analysis/                   # make_figures.py, tables.py
```

- [ ] **Scenario generator** (seeded): true disagreement, stale snapshot, scope or unit mismatch (FY-to-date vs quarter), formatting-only difference, paraphrased metric names, and **no-conflict controls**. Each scenario ships with ground truth.
- [ ] Split scenarios into **dev** (used for tuning and calibration) and **test** (touched only for final numbers). Write the split down.
- [ ] Add at least **one domain you did not author**, built from public spreadsheets and documents.
- [ ] **Fault injector** with fixed schedules (transient, permanent, timeout, silent stale, corrupted, schema change).
- [ ] Implement the baselines B0 to B3 from paper Section 6.3 and the AI-OS ablations (without validation, reconciliation, retry, degrade, replan, gate; overwrite vs supersede; confidence-only and human-only promotion).
- [ ] **Oracle approver** for the human gate in bulk runs; separate small human study later.
- [ ] Metrics code with unit tests (it is easy to get ECE or pass^k subtly wrong).

Phase 3: Run the experiments (about 3 to 4 weeks, mostly waiting)

- [ ] **Pilot first**: 5 scenarios, 2 conditions, 3 runs. Measure tokens and cost per run and fix harness bugs.
- [ ] Budget with: `scenarios x conditions x runs x models x (calls per run x tokens per call)`. If it is too large, run full sweeps on a cheaper model and a smaller test subset on the strongest model, and say so in the paper.
- [ ] Models: at least two hosted and one open-weight served locally (vLLM, Ollama or llama.cpp).
- [ ] At least about 10 runs per scenario per condition (more if variance is high); fixed seeds and temperature; save every trace.
- [ ] Run experiments in this order (each answers one RQ): RQ2 detection, RQ4 confidence, RQ3 resolution, RQ5 faults, RQ6 memory, RQ7 consistency and cost, RQ1 planning.

- [ ] Keep a log of **negative or surprising results**. Report them; they make the paper more credible.

Phase 4: Analysis (about 1 to 2 weeks)

- [ ] 95% bootstrap confidence intervals; paired tests across conditions; report effect sizes, not only p-values.
- [ ] Produce the planned outputs: detection table by conflict class, resolution comparison, reliability diagram and AUROC, fault-by-policy matrix, contamination plot, pass^k curves, overhead table.
- [ ] Error analysis: sample failures and classify them (consider the MAST failure categories as a lens). One page of this is worth more than another table.
- [ ] Revisit hypotheses H1 to H6: state clearly which held and which did not.

Phase 5: Write v2 (about 2 weeks)

- [ ] Replace all `[TBD]`; update abstract and conclusion so they match the real results (do not keep claims the data did not support).
- [ ] **Verify every reference** against the original source; add authors and venues for items marked `[verify]`; re-check the 2026 preprints for final venues.
- [ ] Re-check related work for newer papers on the same ideas (agent memory write gates, conflict reconciliation, fault tolerance). Search arXiv one more time right before submission.
- [ ] Add a short threat model for prompt injection through ingested documents and memory poisoning, and what you did or did not test.
- [ ] Add reproducibility appendix (commit hash, configs, seeds, prompts, cost, licences) and the AI-use statement.
- [ ] Convert to the target template (LaTeX). Ask one or two experienced people to read it before submission; a hostile reader is more useful than a friendly one.

Definition of done

- [ ] Results for every RQ (or an explicit note why one was dropped).
- [ ] Code, scenario generator and traces released and runnable from a clean clone.
- [ ] No placeholders; all citations verified.
- [ ] Employer clearance in hand (section 2).

---

4. Timeline (back-planned from about 1 Feb 2027)

Weeks from 13 Oct	Work
1 to 2	Employer check; Phase 0; start Phase 1
3 to 4	Finish Phase 1; start harness
5 to 7	Finish harness; pilot
8 to 11	Full runs; start analysis
12 to 13	Analysis; write v2
14	Internal review; fix; build the PDF
15	**arXiv submission** (allow time for endorsement)
16	Submit to workshop or venue

Total is about 16 weeks. If you work fewer hours, stretch it rather than skipping Phase 3; the paper stands or falls on those experiments.

---

5. How to publish

Step 1: Preprint on arXiv

1. Create an arXiv account with your **personal** email.
2. Start a new submission. Suggested primary category: **cs.AI**; consider cross-lists **cs.MA** (multiagent systems) and **cs.SE**. Pick the category honestly, since moderators can reclassify.
3. **Endorsement.** As a new, independent submitter you will probably need an endorsement. arXiv's own page says endorsement verifies that contributors belong to the scientific community, and that an institutional email can expedite it. Since 21 Jan 2026, arXiv no longer accepts an institutional email alone as the sole qualifier for new authors. Plan to need a personal endorser.
    - Start the submission first; arXiv emails you an endorsement request containing a six-character code.
    - Send it, with your **finished PDF**, to a small number of eligible authors in the category whom you can reasonably approach (people whose work you cite, colleagues, local researchers, open-source contributors in this area). arXiv says it is inappropriate to email large numbers of potential endorsers or to repeat requests to the same person.
    - Some endorsers publish their own procedures (for example a public pull-request process). Read and follow each person's rules.

    - Do not pay anyone for endorsement.
4. Upload LaTeX source (preferred) or PDF; add the code link (GitHub plus a Zenodo release DOI) in the abstract or first footnote.
5. Expect moderation; keep the paper clearly a research contribution (system plus evaluation), not a product description.

Step 2: Archive the artifacts

☐ Tag a release in GitHub and archive it on **Zenodo** to get a citable DOI for the code and data.
☐ Publish the scenario generator, ground truth and traces alongside it.
☐ Add a `CITATION.cff` file.

Step 3: Peer-reviewed venue

Options, in order of fit for a first paper:

1. **ICLR 2027 workshops.** The main-conference paper deadline (25 Sep 2026) has passed. The ICLR 2027 workshop call lists a suggested workshop-contribution deadline of **1 Feb 2027** and notification by 26 Feb 2027, but each workshop sets its own deadline and format, and the accepted workshop list was not yet public when I checked. Watch the ICLR site for workshops on agents, LLM systems, memory or reliability. Create an **OpenReview profile now**; new profiles can take two weeks or longer to be approved.
2. **Rolling-review venues** (for example TMLR, or ACL Rolling Review for NLP venues): no fixed deadline, but expect a higher bar and a full empirical paper. Verify current requirements on their sites.
3. **Industry or systems tracks and workshops** at other conferences (agent systems, MLOps, enterprise AI); search for calls in the weeks you finish.
4. **Journals** are slower; use only if you want a longer, more complete version later.

Rules to follow for any venue:

- Read the **dual-submission and preprint policy** (most ML venues allow an arXiv preprint, but anonymise the submitted PDF where review is double-blind).
- Follow the exact template and page limit; do not submit to two venues at once unless both allow it.
- Include the AI-use and reproducibility statements the venue asks for.

Step 4: After posting

- Share the arXiv link and repo on relevant communities (Hugging Face, LangChain/LangGraph forums, X/LinkedIn) with a short, honest summary and the key result.
- Respond to issues and questions; reviewers and readers check whether the code runs.

---

6. Questions reviewers will ask (prepare answers)

1. What is new compared with LangGraph supervisors, AutoGen group chat, LLMCompiler, Zep and Mem0?
2. Are the scenarios only your own? (Needs a domain you did not author.)
3. Does it still work with a weaker or local model?
4. How much extra cost and latency does governance add?
5. How do you know the human gate is not doing all the work? (Ablation without the gate.)
6. Is confidence calibrated, and what happens if it is not?
7. What happens when the document content itself is adversarial?
8. Why call it an "OS"?

---

7. Official pages to re-check

- arXiv endorsement help: https://info.arxiv.org/help/endorsement.html
- arXiv policy updates: https://blog.arxiv.org/
- ICLR 2027 workshops call: https://iclr.cc/Conferences/2027/CallForWorkshops
- ICLR 2027 dates and policies: https://iclr.cc/Conferences/2027/Dates